

我的網頁製作流程 SOP

本文件記錄從資料規劃到網頁上線的完整流程,涵蓋使用工具、操作步驟、提示詞範例

與產出物。共分六個階段,依序執行可從零打造一個正式上線的個人網頁專案。

流程總覽

階段 1. 底層資料規劃 — Obsidian

階段 2. 資料分析與統整 — Genspark AI

階段 3. 網頁原型設計 — Claude(Artifacts)

階段 4. 轉換為 React 專案 — VS Code

階段 5. 版本控管與部署 — GitHub + Vercel

階段 6. 上線檢核與後續維護

階段 1. 底層資料規劃 — Obsidian

用途:在動手做網頁之前,先把腦中的想法、要呈現的內容、頁面架構整理成結構化筆記,作為後續所有步驟的「資料來源」。

使用工具:Obsidian(本地 Markdown 筆記軟體,支援雙向連結與圖譜)。

操作步驟:

- a. 建立一個專屬 Vault(例如「我的網頁專案」)。
- b. 用一頁 index.md 列出網頁的主要區塊(關於我 / 作品集 / 部落格 / 聯絡方式等)。
- c. 每個區塊各開一頁筆記,寫下要放的文字、圖片、連結。
- d. 用 [[雙向連結]] 把相關筆記串起來,確認資訊沒有遺漏。

注意事項:這一步只整理「內容」,不要管設計與排版;先求資料完整。

產出物:一個結構清楚的 Obsidian Vault,內含網頁要呈現的所有原始內容。

階段 2. 資料分析與統整 — Genspark AI

用途:把 Obsidian 中較鬆散的筆記交給 AI 做二次整理,讓資料更貼近網頁呈現所需的結構(分類、摘要、亮點抽取)。

使用工具:Genspark AI(具網路檢索與資料統整能力的 AI 工具)。

操作步驟:

- a. 將 Obsidian 對應段落的内容複製貼到 Genspark。
- b. 輸入提示詞請 AI 進行分析與重新組織。
- c. 檢視輸出,將整理後的版本回貼到 Obsidian 對應筆記(或另存為「v2 整理後」分頁)。

提示詞範例:

「請幫我把以下資料做資料分析與整理,輸出為網頁可用的結構(包含區塊標題、條列重點、一句話摘要),語氣維持專業但口語化。」

注意事項:AI 產出需人工審閱,確認沒有事實錯誤或語氣偏離。

產出物:結構化、可直接做為網頁文案的資料版本。

階段 3. 網頁原型設計 — Claude(Artifacts)

用途:把整理好的資料交給 Claude,直接生成可預覽的 HTML/CSS 網頁原型,快速驗證視覺與排版方向。

使用工具:Claude(使用 Artifacts 功能產生可即時預覽的網頁)。

操作步驟:

- a. 將步驟 2 整理後的資料貼進對話。
- b. 描述網頁風格(例如:極簡、深色背景、卡片式排版、配色)。
- c. 請 Claude 產出單頁 HTML(含內嵌 CSS)做為原型。
- d. 在 Artifacts 預覽,逐區塊回饋修改(字體、間距、區塊順序)。

提示詞範例:

「根據以下資料,設計一個單頁個人網頁。風格:深色、極簡、卡片排版。包含區塊:Hero、關於我、作品集、聯絡方式。請輸出完整 HTML 並用 Artifacts 預覽。」

注意事項:這一步只是原型,先確認版面與內容呈現方式,還不需要在意元件化或互動效果。

產出物:一份可預覽的 HTML 原型(含完整文案與基本樣式)。

階段 4. 轉換為 React 專案 — VS Code

用途:將靜態 HTML 原型重構為元件化的 React 專案,方便後續維護、擴充與部署。

使用工具:VS Code(編輯器)、Node.js、Vite 或 Create React App、Git。

操作步驟:

- a. 在本機建立 React 專案:`npm create vite@latest my-site -- --template react`。
- b. 將步驟 3 的 HTML 原型貼到 Claude / Copilot,請它拆成 React 元件 (Header、Hero、About、Projects、Contact、Footer)。
- c. 在 VS Code 中建立對應的 .jsx 檔,逐一貼入並調整 import / props。
- d. 把 CSS 改為 CSS Modules 或 Tailwind(視個人偏好)。
- e. 本機 `npm run dev` 預覽,逐元件除錯。

提示詞範例:

「請把以下 HTML 拆成 React 元件,每個元件一個檔案,使用函式元件 + Hooks,樣式用 CSS Modules。」

注意事項:轉換時保留原型的視覺,不要在這一步同時改設計,避免一次變動太多難以除錯。

產出物:可在本機 `npm run dev` 跑起來的 React 專案。

階段 5. 版本控管與部署 — GitHub + Vercel

用途:把專案推上 GitHub 做版本控管,並透過 Vercel 自動部署上線,之後每次 push 都會自動更新網站。

使用工具:Git、GitHub、Vercel。

操作步驟:

- a. 在 VS Code 終端機初始化:`git init` → `git add .` → `git commit -m "init"`。
- b. 在 GitHub 建立新 Repo,將本機專案推上去:`git remote add origin <repo-url>` → `git push -u origin main`。
- c. 登入 Vercel,選擇「Import Project」並連接該 GitHub Repo。
- d. 確認 Framework Preset 為 Vite(或 CRA),Build Command 與 Output Directory 通常自動偵測。
- e. 點選 Deploy,等待約 1–2 分鐘完成,取得 `.vercel.app` 網址。

注意事項:確認 `.gitignore` 已排除 `node_modules`;敏感資訊(API Key)使用 Vercel 的 Environment Variables,不要直接寫進程式碼。

產出物:一個可公開存取的網址,並建立「push 即自動部署」的工作流。

階段 6. 上線檢核與後續維護

用途: 確認網站在不同裝置與瀏覽器上正常運作,並建立後續更新節奏。

檢核清單:

- a. 行動裝置開啟,確認 RWD(響應式設計)正常。
- b. 用 Chrome DevTools Lighthouse 跑一次效能、SEO、無障礙分數。
- c. 檢查所有連結、按鈕、表單是否可正常運作。
- d. 自訂網域(如有):在 Vercel 設定 Domain,於網域商設定 DNS。
- e. 加上基本 SEO:<title>、meta description、Open Graph 圖片。

後續維護:

- a. 新內容先回到 Obsidian 撰寫,再依步驟 2-4 流程同步到網站,保持「資料來源唯一」原則。
- b. 重大改版前先在 Git 開新分支,確認沒問題再合併到 main。

產出物: 正式可對外公開、可持續維護的個人網頁。